AP CSA: Key Concepts Checklist

High-Yield Exam Topics · Static vs. Non-Static #01 Top Priority · Code & Traps

[ ] 01. Static vs. Non-Static (Class vs. Instance) — TOP PRIORITY

static belongs to the Class as a whole with ONE shared copy in memory. Non-static belongs to an individual object created with new.

- static methods/fields are called via ClassName.method() or ClassName.field
- Non-static methods/fields are called via objectName.method()
- static methods CANNOT use "this" or access instance fields directly
- Calling an instance method from static main without creating an object causes "non-static method cannot be referenced from a static context"

```
public class Student {
  private String name;           // Instance var (unique per student)
  private static int count = 0;  // Static var (shared across all students)
  public Student(String n) { name = n; count++; }
  public static int getCount() { return count; }
```

[ ] 02. Inheritance, super(), & Polymorphism (Dynamic Binding)

Subclasses inherit superclass methods via extends. super() calls the superclass constructor and MUST be the first line of the subclass constructor.

- Compile-time check: Compiler verifies method exists on declared reference type
- Runtime execution: Java runs the overridden method on the actual instantiated object class (Dynamic Binding)
- Subclasses cannot access private superclass fields directly; must use getters/setters or super()

```
Animal a = new Dog(); // Polymorphic declaration
a.speak();            // Runs Dog.speak() at runtime
// a.fetch();         // CE: Animal reference does not declare fetch()
```

[ ] 03. References, Aliasing, & == vs. .equals()

Primitives store raw values. Objects store memory addresses. == on objects checks memory location identity, NOT character content equality.

- Always use .equals() to compare String contents
- Aliasing: Dog d2 = d1; does NOT create a second dog; both variables point to the same object in heap memory
- Strings are immutable; methods like .toLowerCase() return a NEW String and never modify the original

```
String s1 = new String("hi");
String s2 = new String("hi");
System.out.println(s1 == s2);     // false (different heap addresses)
System.out.println(s1.equals(s2)); // true  (same characters)
```

[ ] 04. 1D Arrays vs. ArrayList<E> & The Backward Removal Loop

Arrays have fixed capacity (arr.length, field). ArrayList has dynamic size (list.size(), method) and stores wrapper objects.

- Arrays: arr[i] | ArrayList: list.get(i), list.set(i, val)
- Removing elements in a forward loop shifts indices, skipping consecutive matches
- Always traverse backwards (for int i = list.size() - 1; i >= 0; i--) when removing elements

```
// Safe removal loop:
for (int i = list.size() - 1; i >= 0; i--) {
  if (list.get(i).equals("remove")) list.remove(i);
}
```

[ ] 05. 2D Arrays & Row-Major Traversal

2D arrays are arrays of arrays stored in row-major order: grid[row][col]. Number of rows = grid.length, number of cols = grid[0].length.

- Outer loop indexes rows (r = 0; r < grid.length; r++)
- Inner loop indexes columns (c = 0; c < grid[r].length; c++)

```
int[][] matrix = new int[2][3]; // 2 rows, 3 columns
for (int r = 0; r < matrix.length; r++)
  for (int c = 0; c < matrix[r].length; c++)
    matrix[r][c] = r * 10 + c;
```

[ ] 06. Recursion & Base Case Tracing

A recursive method calls itself with simpler input until reaching a base case. Missing or unreachable base cases trigger StackOverflowError.

- Code before recursive call executes on the way DOWN the stack
- Code after recursive call executes on the way BACK UP the stack

```
public static int mystery(int n) {
  if (n <= 1) return 1; // Base case
  return n + mystery(n - 1); // mystery(4) = 4+3+2+1 = 10
}
```

[ ] 07. Search & Sort Algorithms (Big-O & Mechanics)

AP CSA requires understanding Sequential Search, Binary Search, Selection Sort, Insertion Sort, and Merge Sort.

- Binary Search (O(log n)) REQUIRES SORTED DATA! Halves search space each step
- Selection Sort (O(n^2)): Repeatedly finds minimum and swaps to front (minimum swaps)
- Insertion Sort: O(n) best / O(n^2) worst (fast if mostly sorted)
- Merge Sort (O(n log n)): Recursive divide-and-conquer; merges sorted halves

```
// Binary search middle formula: mid = (low + high) / 2;
// If target < arr[mid], search left: high = mid - 1;
```

[ ] 08. Boolean Logic, Short-Circuiting, & De Morgan's Laws

!(A && B) is equivalent to (!A || !B). !(A || B) is equivalent to (!A && !B). Short-circuit evaluation stops compound checks early.

- Negation of < is >= (must include equality!)
- Short-circuit: in (s != null && s.length() > 0), if s is null, s.length() is never called

```
// Safe null check with short-circuit &&:
if (str != null && str.length() > 0) { ... }
```